Sri Sivasubramaniya Nadar College of Engineering
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Computer Science and Engineering

Degree & Branch	Five Year Integrated M.Tech (CSE)		**Semester**	IV
Subject Code & Name	ICS1405 – Software Engineering Principles and Practices [TCP]			
Assignment - 5	**Explore Git and GitHub commands and Practice Source code management**			
Academic Year	2025-2026 EVEN	**Batch**	2024-2029	**Regulations – R2023**
MINI PROJECT	**NATHANIEL CHRISTIAN - 3122247001037** **SHARRUK - 3122247001061** **NAVIN KUMAR - 3122247001039**			**Date: 27-02-2026**

FlowMotion: Intelligent Habit & Routine Management System

1. Problem Analysis

Modern software projects evolve continuously and involve frequent changes across multiple modules. The FlowMotion project, an intelligent habit and routine management system, includes backend logic, user interface components, and AI-based recommendation utilities. Without proper source code management, such projects become difficult to maintain, debug, and extend.

This experiment addresses the need for a structured version control mechanism using **Git and GitHub** to manage changes, track history, support parallel development, and ensure maintainability of the FlowMotion project.

2. Objectives

- To create and manage a Git repository for the FlowMotion project
- To practice essential Git commands for version control
- To implement a structured branching workflow
- To analyze the effectiveness of the chosen Git workflow
- To document commit history, branching strategy, and merge operations

---

3. Tools and Environment

- **Version Control Tool:** Git
- **Remote Repository Hosting:** GitHub
- **Development Environment:** Replit / Visual Studio Code
- **Programming Language:** Python
- **Framework:** Django
- **Operating System:** Linux

---

4. Repository Setup

4.1 Local Repository Initialization

The FlowMotion project directory was initialized as a Git repository using:

```
git init
```

This created the `.git` directory to track version history.

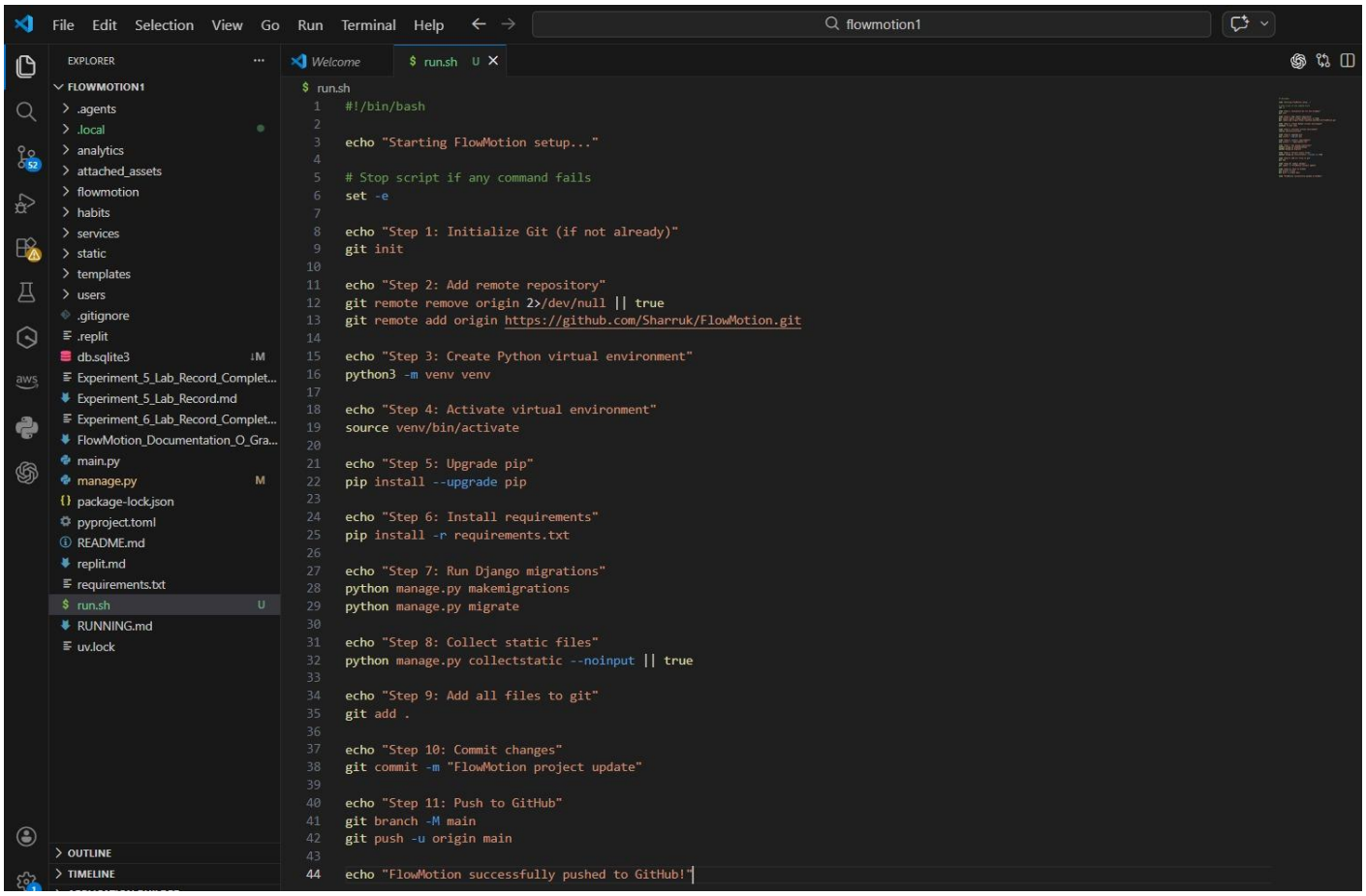
4.2 Linking with Remote Repository

A remote repository was created on GitHub and linked to the local repository:

```
git remote add origin https://github.com/<username>/FlowMotion.git
git remote -v
```

---

5. Branching Strategy



The project follows a **Feature Branch Workflow**, ensuring that the main branch always contains stable and deployable code.

Branches Used

- **main** – Stable production-ready code
- **develop** – Integration branch for features
- **feature/ai-recommendation** – AI recommendation logic
- **feature/ui-dashboard** – UI enhancements
- **bugfix/login-issue** – Authentication-related fixes

Branch Hierarchy (Textual Representation)

```
main
 └── develop
      ├── feature/ai-recommendation
      └── feature/ui-dashboard
```

---

6. Repository Structure

```
FlowMotion/
├── .gitignore
├── flowmotion/        (Project configuration)
├── habits/            (Habit logic & AI utilities)
├── users/             (Authentication & profiles)
├── templates/         (HTML templates)
├── static/            (CSS and JavaScript files)
└── manage.py
```

The `.gitignore` file excludes unnecessary files such as virtual environments, compiled files, and database files.

---

7. Version Control Operations

7.1 Staging and Committing Changes

```
git add .
git commit -m "Initial commit: Django project setup"
```

Incremental commits were made for each logical change:

```
git commit -m "feat: add habit tracking models"
git commit -m "feat: integrate AI recommendation utility"
git commit -m "fix: resolve dashboard rendering issue"
```

7.2 Viewing Commit History

```
git log --oneline --graph
```

This command helped visualize commit history and branching relationships.

---

8. Branch Merging and Conflict Resolution

While merging `feature/ui-dashboard` into `develop`, a conflict occurred in an HTML template due to overlapping changes.

Conflict Resolution Steps

1. Conflict detected during merge:

   ```
   git merge feature/ui-dashboard
   ```

2. Conflicting file manually edited to retain both changes
3. Resolved file staged and committed:

   ```
   git add templates/base.html
   git commit -m "fix: resolve merge conflict in navigation bar"
   ```

---

9. Workflow Implementation

```
natc@JFlenovoNC:/mnt/c/Users/natha/Downloads/flowmotion1/flowmotion1$ ./run.sh
Starting FlowMotion setup...
Step 1: Initialize Git (if not already)
Reinitialized existing Git repository in /mnt/c/Users/natha/Downloads/flowmotion1/flowmotion1/.git/
Step 2: Add remote repository
Step 3: Create Python virtual environment
Step 4: Activate virtual environment
Step 5: Upgrade pip
Requirement already satisfied: pip in ./venv/lib/python3.12/site-packages (24.0)
Collecting pip
  Downloading pip-26.0.1-py3-none-any.whl.metadata (4.7 kB)
Downloading pip-26.0.1-py3-none-any.whl (1.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 8.1 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 24.0
    Uninstalling pip-24.0:
      Successfully uninstalled pip-24.0
Successfully installed pip-26.0.1
```

Development Workflow

1. Pull latest changes

   ```
   git pull origin develop
   ```

2. Create feature branch

   ```
   git checkout -b feature/new-feature
   ```

3. Implement changes with incremental commits
4. Push branch to GitHub

   ```
   git push origin feature/new-feature
   ```

5. Merge into develop after review

---

10. Workflow Analysis

Efficiency

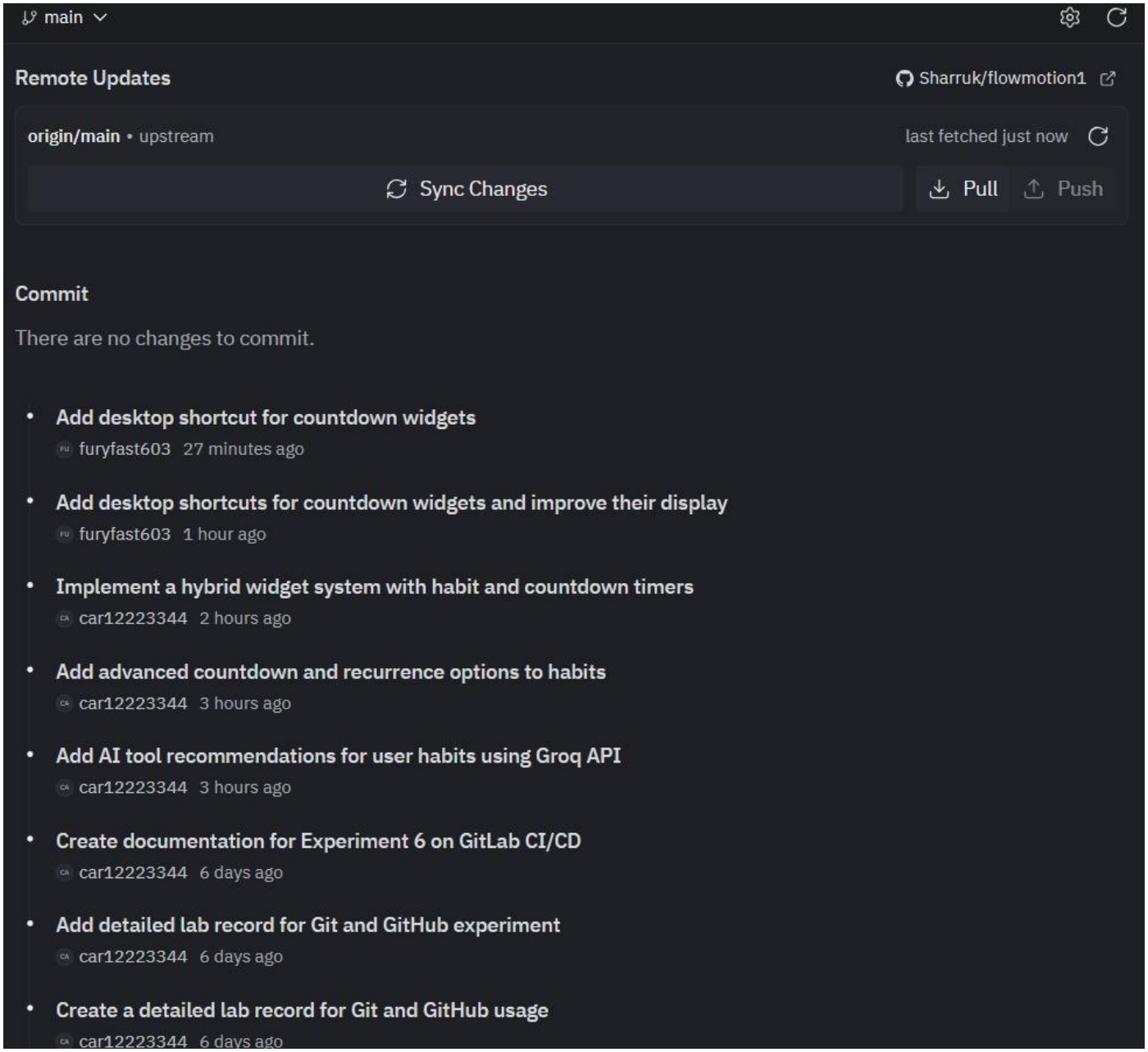
- Enabled parallel development of AI logic and UI components
- Reduced risk of breaking stable code
- Improved traceability of changes

Maintainability

- Clear commit history helped understand code evolution
- Modular branches isolated experimental features

Risk Management

- Conflicts were limited to specific branches
- Main branch remained stable throughout development



11. Comparison of Alternative Workflows

Workflow	Advantages	Disadvantages	Suitability
Centralized	Simple	High conflict risk	Low
GitFlow	Structured	Complex	Moderate
Feature Branch	Modular & safe	Needs discipline	High

The Feature Branch Workflow was most suitable for the FlowMotion project.

12. Observations

- Incremental commits made debugging easier
- `.gitignore` prevented accidental commit of sensitive files
- Branching reduced interference between team members

13. Conclusion

This experiment successfully demonstrated the use of Git and GitHub for effective source code management. By implementing a Feature Branch Workflow, the FlowMotion project achieved better organization, improved maintainability, and reduced integration risks. Version control proved essential in managing the continuous evolution of the software.

14. Learning Outcomes

- Practical understanding of Git commands
- Experience with branching and merging strategies
- Ability to resolve merge conflicts
- Awareness of workflow selection in software projects